



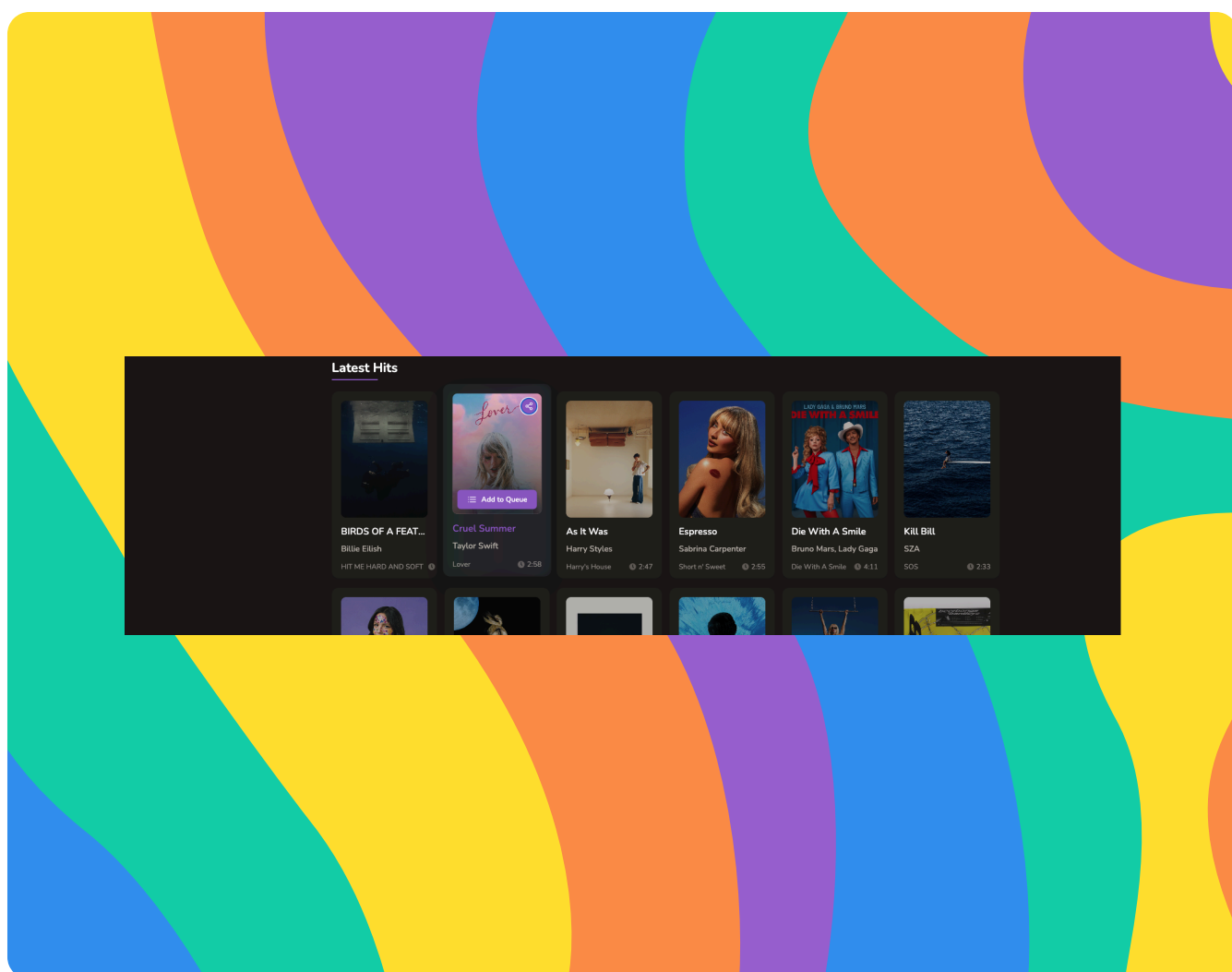
nextwork.org

Build a New App Feature With Claude Code



tahirgroot@gmail.com

<https://medium.com/@tahirbalarabe2/how-to-m...>





Introducing Today's Project!

I'm going to build a share track feature with a button, clipboard copy of the Spotify URL, and a toast notification using Claude Code in my terminal. NextSound is a Spotify clone that already lets me browse tracks from popular artists, search for music, and switch between dark and light themes.

Key tools and concepts

The key tools I used include Claude Code in my terminal, Node.js and npm for dependencies, the `/init` command to analyze my codebase, plan mode with `Shift+Tab`, the `/model` and `/cost` commands, and custom commands like `/staged_plan`. Key concepts I learned include `CLAUDE.md` as an AI memory file, token based pricing for different models like Haiku, Sonnet, and Opus, spending limits, staging feature development in three phases, clipboard API, toast notifications, and automating workflows with custom markdown commands.

Choosing the right AI model

The Claude model I'm using in this project is Sonnet because it gives balanced performance and cost, with detailed explanations perfect for learning. Other model options include Haiku for faster simple tasks and Opus for complex reasoning. I would choose a more powerful model like Opus if I were solving a difficult logic problem or refactoring a huge codebase. I would choose a simpler model like Haiku if I only needed quick formatting or minor text edits without deep analysis.

Challenges and wins

This project took me approximately 90 minutes for the main feature plus another 30 minutes for the secret mission. The most challenging part was understanding how to properly set up the budget and API key in the Anthropic Console before Claude Code would work.



tahirgroot@gmail.com

NextWork Student

nextwork.org

It was most rewarding to see the custom `/staged_plan` command automate my entire planning workflow and build the multi share feature in minutes instead of manually prompting each stage. With the skills I learned, I want to build next a full CRUD app with Claude Code handling the database integration and toast notifications for every action.



Setting Up My Project

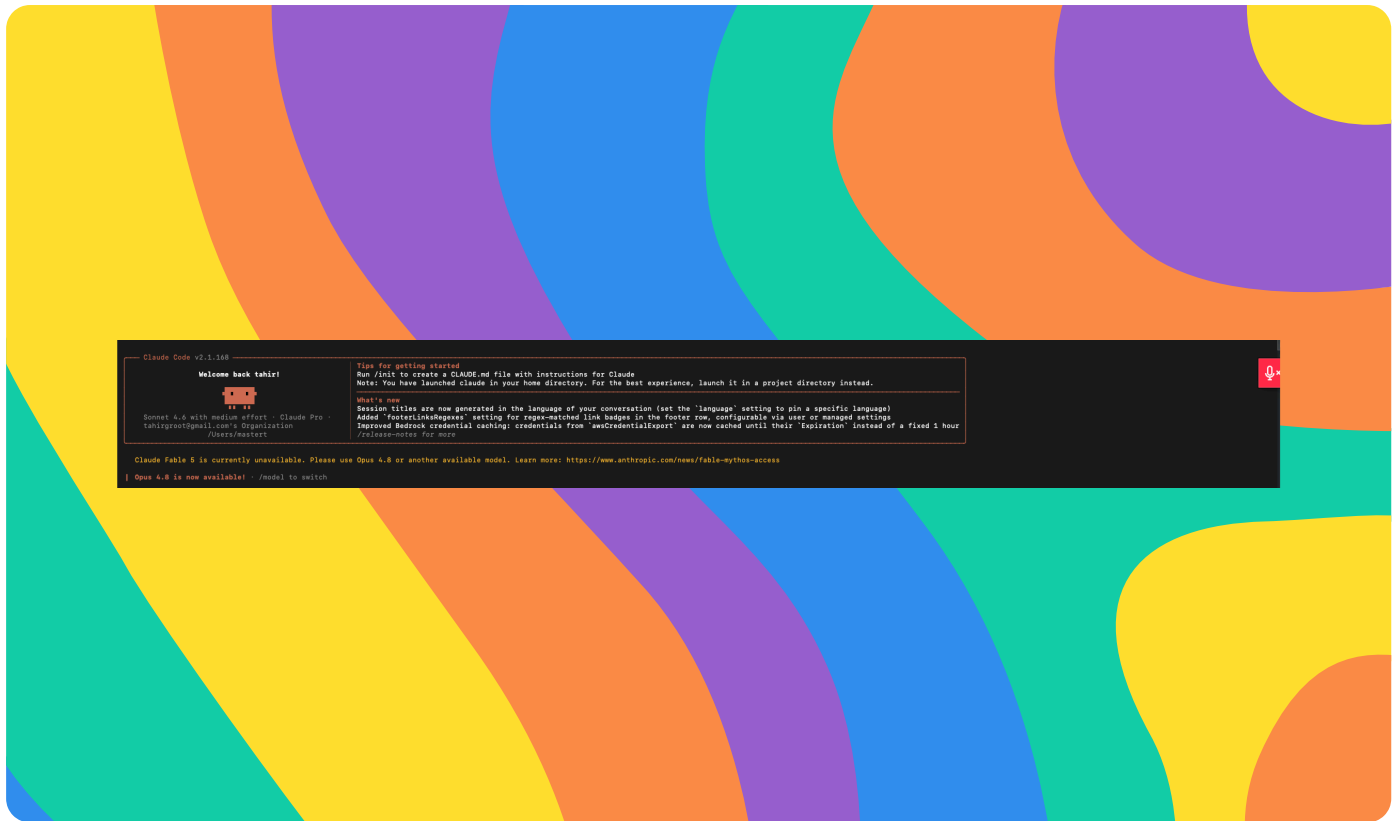
In this step, I'm setting up my coding environment by cloning NextSound's code from GitHub, installing Node.js as the runtime to run Claude Code, and then installing Claude Code via npm. I'm creating my Claude account on console.anthropic.com because I need an API key and to add a minimum \$5 credit, which lets Claude Code access my entire codebase and edit files directly from my terminal.

The Web App Code

Before I could start running the NextSound web app, I ran `npm install` to download and set up all the external libraries and tools that the project needs, like React and other dependencies listed in the `package.json` file. npm is a package manager for JavaScript and Node.js that reads the `package.json`, fetches the required packages from the npm registry, and stores them in a `node_modules` folder so the app can use them properly.

Claude Code

Claude Code is a command line AI tool that runs directly in my terminal with access to my entire codebase. Unlike AI chats in my browser, Claude Code can edit files, run terminal commands like `npm install` or `git status`, understand project context immediately, and execute shell scripts without me copying and pasting code back and forth.





Managing Costs

Why budget management matters

I set up a \$5 budget in my Anthropic Console under Spend Limits to prevent accidentally spending more than I intend. AI models bill based on tokens, meaning each prompt and response has a variable cost depending on length and model choice like Sonnet or Opus. So if I didn't have a budget, a long debugging session or an infinite loop of requests could unexpectedly run up my API bill without any automatic stop, leaving me with charges I didn't plan for.

Using the /cost command

The /cost command shows my token usage and estimated costs for the current conversation, breaking down input and output tokens. It's good practice to run /cost when I finish a feature or before switching tasks, so I can monitor my spending and stay within budget, especially since different models like Haiku, Sonnet, or Opus have different per token prices.



NextWork Student

nextwork.org

Spend limits

\$10.05 spent

Resets Jul 1, 2026

\$5

Email notifications

Adjust limit

Edit



Initializing Claude in My Project

In this step, I'm going to install the NextSound project's dependencies so everything runs smoothly. Then I'll initialize Claude Code inside my project folder so it can analyze my codebase structure. Finally, I'll manage Claude's memory by setting custom instructions, which helps Claude remember my preferences and follow my coding style across sessions.

The /init command

The /init command makes Claude analyze my entire codebase, understand the project structure, libraries, and dependencies, then generate a CLAUDE.md file that serves as Claude's memory for future sessions. This helps Claude remember my project's architecture, coding standards, and rules so I don't have to re explain everything each time I start a new chat.

CLAUDE.md

CLAUDE.md is a memory file, which means it stores project specific instructions and context for Claude Code to reference automatically in every session. My project's CLAUDE.md file has multiple sections, covering codebase architecture, file structure, coding standards, preferred libraries, build commands, and notes on how components connect. This is different from a README.md because a README is written for human developers to understand what the project does and how to use it, while CLAUDE.md is written for the AI to follow rules and maintain consistency when helping me code.



tahirgroot@gmail.com

NextWork Student

nextwork.org

`## Rules`

`- Do not implement new features that haven't been explicitly requested. If you'd like to add something not discussed, confirm the plan first and wait for approval before writing any code.`



Managing memory

Updating memory via the terminal

To add a new instruction, I typed a hash symbol (#) in the Claude Code prompt, then pasted the rule into memory mode. The new instruction tells Claude Code not to implement features I didn't ask for, and to confirm any extra additions with me first. The memory is saved in CLAUDE.md as project memory.

Claude manages different types of memory

Project memory is useful for setting rules specific to the current codebase that my whole team should follow. For example, "Always use the existing button component instead of creating new ones." User memory is useful for personal preferences that apply across every project on my computer. For example, "Always explain code in simple terms with comments" or "Never suggest external API calls without asking me first."



Using Plan Mode

In this step, I'm going to run the NextSound web app to see how it works, then create a three stage plan with Claude Code for the share feature: first adding the share button UI, second implementing clipboard copy functionality, third showing a toast notification. I'll review Claude's plan, give feedback, and implement each stage one at a time.

Understanding plan mode

Claude Code's plan mode is useful for creating a detailed implementation strategy without writing any code yet. I activated plan mode by pressing Shift + Tab in the terminal. Without plan mode, I would have had to let Claude write code immediately, risking mistakes or unwanted changes before I could review the approach. Plan mode let me review, adjust, and add stage by stage controls safely first.

Refining the plan to add stage controls

I used plan mode to build a three stage share feature without letting Claude write code immediately. I refined the plan by adding strict instructions that Claude must stop after each stage and wait for my confirmation before moving to the next stage, because I wanted to test each part fully before building on top of it and avoid compounding errors.



```
• Here's the plan in full:

---
Stage 1 - Share Button UI (console.log only)

Goal: Add a Share2 icon button to each track card's hover overlay. On click, fires console.log('[Share]', title, spotifyUrl) - no other side effects.
Files: src/components/ui/TrackCard.tsx only.
Gate: Stop and wait for approval before Stage 2.

---
Stage 2 - Clipboard Copy

Goal: Replace the console.log with navigator.clipboard.writeText(track.external_urls.spotify). Silently skips if no URL is present.
Files: src/components/ui/TrackCard.tsx only.
Gate: Stop and wait for approval before Stage 3.

---
Stage 3 - Toast Notification

Goal: After the clipboard write succeeds, show a "Link copied!" toast that auto-dismisses after 2 seconds.
Files created: src/components/ui/Toast.tsx, src/context/toastContext.tsx
Files modified: src/main.tsx (wrap in <ToastProvider>), src/components/ui/TrackCard.tsx (call showToast)
Toast: fixed bottom-center, Pumpkin orange background, framer-motion slide-up/fade animation.
```



The Complete Feature

The three-stage implementation

The first stage starts the build by creating a share button UI on each track card and logging a debug message to the console when clicked. The second stage builds on Stage 1 by implementing clipboard functionality so clicking the button copies the track's Spotify URL. The third stage adds a toast notification that confirms the URL was copied, giving the user clear feedback.

Benefits of staged development

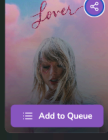
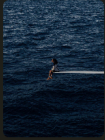
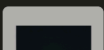
Breaking a feature build into stages helped because I could test each piece before moving to the next, making bugs easier to catch and fix. After each stage, I tested Claude Code's work by refreshing the browser, clicking the share button, checking the console for debug messages, verifying the clipboard copy, and confirming the toast notification appeared.

How much it costed to build this feature

After building the share feature, I ran `/cost` and saw my total cost was \$1.11 for this session. Claude Sonnet 4.5 handled most of the work with over 2 million cache reads and cost \$1.04, while Haiku 4.5 contributed smaller amounts. I added 196 lines of code and removed 5 lines. The session used 9% of my weekly limit, and 48% of my usage came from subagent heavy sessions, while 28% came from running `/init`.



Latest Hits

 BIRDS OF A FEAT... Billie Eilish HIT ME HARD AND SOFT	 <i>Cruel Summer</i> Taylor Swift Lover 2:58	 As It Was Harry Styles Harry's House 2:47	 Espresso Sabrina Carpenter Short n' Sweet 2:55	 Lady Gaga & Bruno Mars DIE WITH A SMILE Bruno Mars, Lady Gaga Die With A Smile 4:11	 KILL BILL SZA SOS 2:33
					



Automating My Workflow

Building custom commands

In this secret mission, I'm learning to build features at twice the speed using custom commands. While staged planning is a great way to build features, it takes time to manually activate plan mode, write the three stage prompt, ask for strict controls, and move through each stage. I'm using commands to automate that entire workflow with a single shortcut like `/staged_plan`. This will help me skip repetitive typing, keep my process consistent, and focus more on building rather than prompting.

Use cases of commands

I could create other custom commands for automatically reviewing my code for bugs with `/review`, generating test cases and checking test coverage with `/test`, running a security audit on dependencies with `/security`, or even a `/fix` command that reads an error message and suggests a fix. I could also make `/doc` to generate inline comments and a README update, or `/clean` to remove unused imports and variables.



```
.cursor > commands > staged_plan.md > # Staged Planning Command > ## Example:
1 # Staged Planning Command
2
3 You will use a multi-stage planning approach for the following feature request.
4
5 ## Instructions:
6
7 1. Create a comprehensive 3-stage plan that breaks down the feature implementation:
8     - Stage 1: Basic structure/UI only (no functionality)
9     - Stage 2: Core functionality (no edge cases or polish)
10    - Stage 3: Polish, edge cases, and final touches
11
12 2. For each stage, explain:
13     - What will be implemented
14     - Which files will be created or modified
15     - How to test that stage works correctly
16     - What the next stage will add
17
18 3. Wait for explicit confirmation before implementing each stage. After completing a stage, stop and ask the user to verify it works before moving to the next stage.
19
20 4. After presenting the plan, ask: "Should I proceed with implementing Stage 1?"
21
22 ## Example:
23
24 For a share track feature:
25 - Stage 1: Add share button UI to track cards with console.log on click
26 - Stage 2: Implement clipboard functionality to copy track's Spotify URL in the backend
27 - Stage 3: Add toast notification and handle edge cases
28
29 ---
30
31 Feature Request:
```

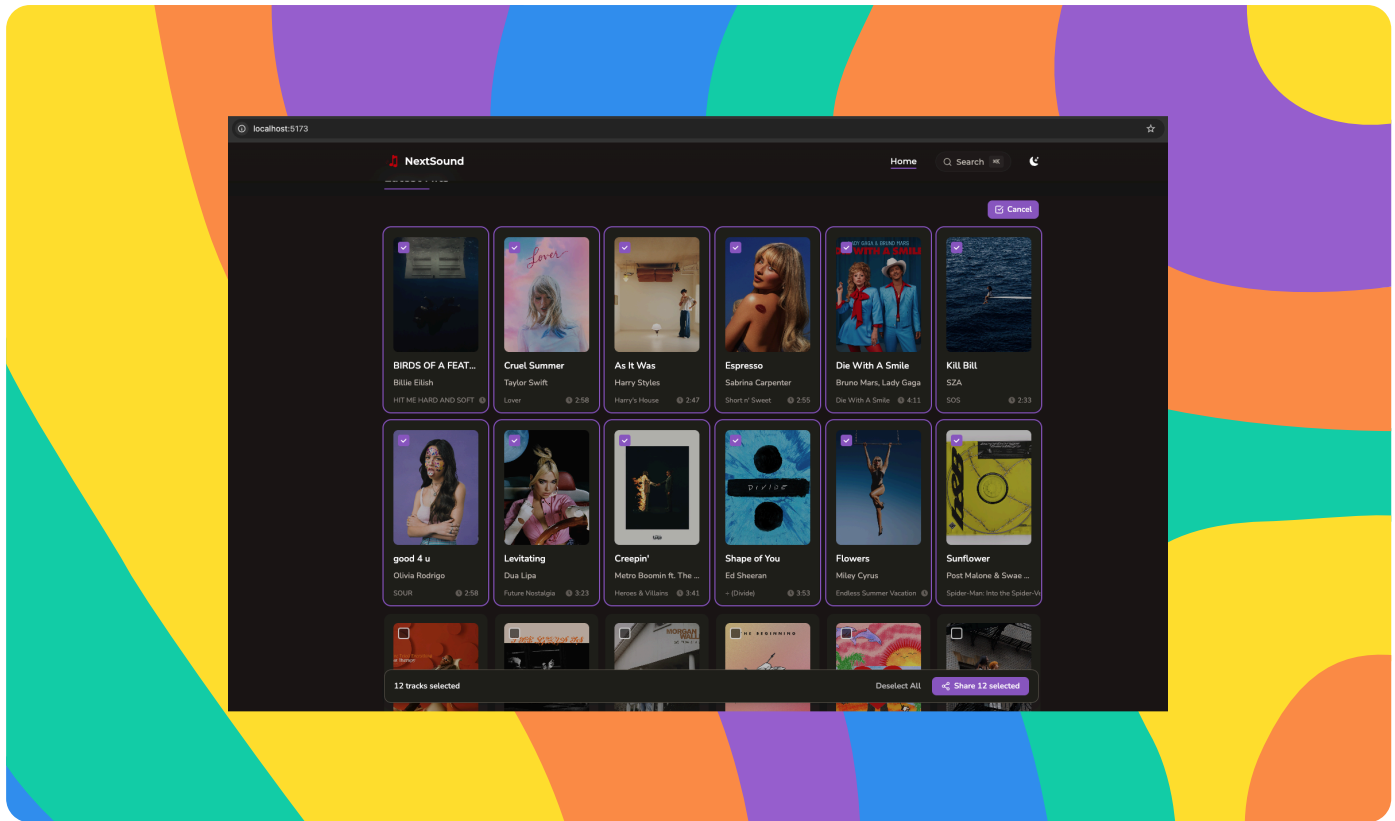



How I Used Commands

A new feature I'm building is a multi share feature that lets users select and share multiple tracks from the latest hits section. Claude broke down the feature into three stages: Stage 1 = add selection mode UI with a "Select Multiple" button and checkboxes on each track, Stage 2 = implement bulk clipboard copy to copy all selected tracks' Spotify URLs at once, Stage 3 = add visual feedback like a toast showing how many tracks were shared and a way to exit selection mode.

Workflow improvements

Compared to planning changes manually, the custom command saved time by automating the entire staged planning workflow. Instead of manually activating plan mode, writing the three stage prompt, adding strict stage controls, and asking to proceed each time, I just typed `/staged_plan` followed by my feature description. This meant no repetitive typing, no risk of forgetting a step, and consistent plans every time. The command handled the plan mode activation, stage breakdown, and controls automatically, letting me focus on testing and building rather than prompting.





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

